Práctica 2.2
Modelos para casos reales.

Docente: Gustavo Landfried

Inferencia Bayesiana Causal 1
2do cuatrimestre 2026
UNSAM

Índice

1. Significancia estadística de los experimentos causales. 2
1.1. Aproximación . . . 4

2. Experimento con datos escasos y corrompidos. 5
2.1. Cómo comparar modelos . . . 6
2.2. Modelo causal alternativo . . . 7

3. Evaluación de tests diagnósticos sin gold standard. 8
3.1. Introducción . . . 8
3.2. Modelo 0: una población y un test (no identificable). . . . 8
3.3. Modelo 1 "Hui-Walter": varias poblaciones y tests. . . . 9
3.4. Modelo 2 "Dendukuri": con una covarianza. . . . 10
3.4.1. Evaluación del modelo Dendukuri . . . 12
3.5. Modelo 3 "Harmony": con varias covarianzas. . . . 12
3.5.1. Evaluación teórica del modelo Harmony . . . 13
3.5.2. Evaluación empírica del modelo Harmony . . . 13
3.6. Modelo 4: Dendukuri para todos los pares. . . . 14
3.6.1. Evaluación en simulaciones . . . 15
3.7. Datos simulados . . . 18

4. Entrega 18

1. Significancia estadística de los experimentos causales.

¿Cuántas ventas produce una campaña publicitaria? Para responder esta pregunta se realizan experimentos causales. Se divide aleatoriamente a la población en un grupo tratamiento y control y se observa la cantidad de conversiones, n_T y n_C respectivamente. Dado que el desempeño en tratamiento y en control puede cambiar en el tiempo, las empresas realizan experimentos todos los meses. Para evaluar la significancia estadística se suelen usar métodos estadísticos clásicos, no bayesianos. El problema que tienen es que los resultados no suelen ser significativos.

En la figura 1 ofrecemos la especificación completa del modelo bayesiano básico. La cantidad de conversiones n_T y n_C se modelan en base al tamaño de los grupos N_T y N_C y a la probabilidad oculta de conversión q y p. El modelo agrega una variable oculta adicional, d, que representa la diferencia de desempeño entre control y tratamiento.

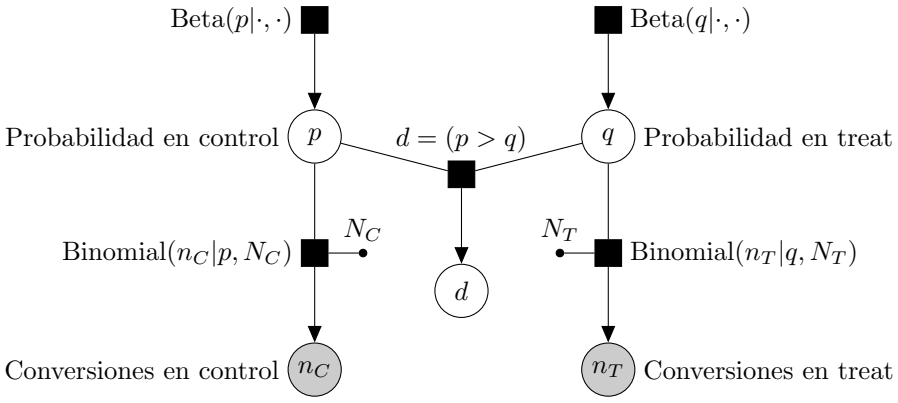


Figura 1: Modelo bayesiano base para estimar efectos causales y su significancia estadística.

Notar que el modelo tiene solución analítica para estimar la probabilidad de tratamiento y de control debido a que la distribución Beta es conjugada con la Binomial. Por lo tanto, si el prior es Beta(α, β) el posterior es Beta($\alpha + n, \beta + (N - n)$).

En el archivo `01-Incrementalidad.csv` van a encontrar los datos de un experimento realizado durante los meses de enero hasta septiembre del 2025. En la siguiente figura se muestra la estimación del desempeño en tratamiento y control respectivo a enero usando el modelo base.

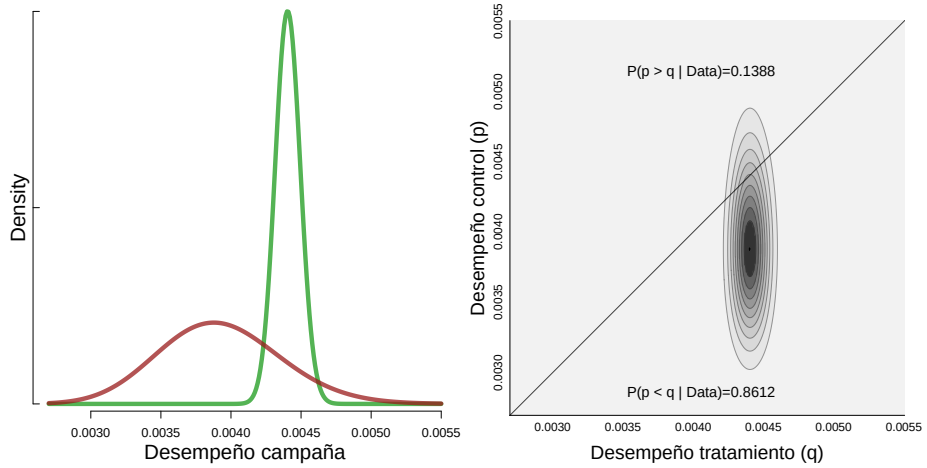


Figura 2

En el gráfico de la izquierda mostramos el posterior del desempeño de control (rojo) y tra-

tamiento (verde). En la figura derecha la probabilidad conjunta, construida como el producto cartesiano entre ambos posteriors. Notar que la probabilidad de que el desempeño de tratamiento sea mayor a de control es de aproximadamente 0,86, lo cual no resulta ser significativo ni concluyente. Si repiten el procedimiento con cada uno de los meses individuales, van a ver que ocurre algo parecido.

El problema principal es que la empresa está analizando los datos por separado. Para resolver este problema vamos a proponer un modelo causal temporal, en el que el desempeño en un cierto tiempo depende del desempeño en el tiempo anterior. De esa forma, los datos de todos los meses quedan conectados en un único modelo, lo que nos va a permitir obtener mejores estimaciones de los desempeños. La figura 3 es la especificación matemática de la distribución conjunta del modelo temporal. Una alternativa es suponer que el desempeño cambia de forma gradual, agregando un poco de incertidumbre en cada paso temporal. En el contexto de estimación de habilidad, donde se usan variables gaussianas, el modelo temporal se especifica directamente como $P(p_t|p_{t-1}, s) = \mathcal{N}(p_t|p_{t-1}, s^2)$, donde s es el ruido. El análogo para la distribución Beta es una Beta centrada en el desempeño del tiempo anterior,

$$P(p_t|p_{t-1}, \kappa) = \text{Beta}\big(p_t \,|\, \kappa\, p_{t-1},\ \kappa\,(1 - p_{t-1})\big) \tag{1}$$

cuya media y varianza son,

$$\mathbb{E}[p_t|p_{t-1}, \kappa] = p_{t-1} \quad , \quad \text{Var}(p_t|p_{t-1}, \kappa) = \frac{p_{t-1}(1 - p_{t-1})}{\kappa + 1} \tag{2}$$

El parámetro de concentración κ cumple el rol inverso del ruido s^2. Si κ es grande, el desempeño de un mes es casi idéntico al del mes anterior. Si κ es chico, el desempeño de un mes es casi independiente del anterior. Como a priori el valor de κ puede estar en cualquier orden de magnitud, le asignamos una distribución uniforme en escala logarítmica, $\log_{10} \kappa \sim U[0, 7]$. Hay un único κ para todos los pasos temporales. La distribución conjunta del modelo es,

$$P(\kappa, p_{0:T}, n_{0:T}) = P(\kappa)\,\text{Beta}(p_0|1, 20)\prod_{t=1}^{T}\text{Beta}\big(p_t \,|\, \kappa\, p_{t-1},\ \kappa\,(1 - p_{t-1})\big)\prod_{t=0}^{T}\text{Binomial}(n_t|p_t, N_t) \tag{3}$$

La figura 3 muestra su especificación gráfica. Para el experimento se usan dos cadenas independientes, una para el desempeño de control p_t y otra para el de tratamiento q_t, cada una con su propio κ. Este modelo no tiene solución analítica, pues la Beta del tiempo anterior integrada contra el kernel no es una Beta.

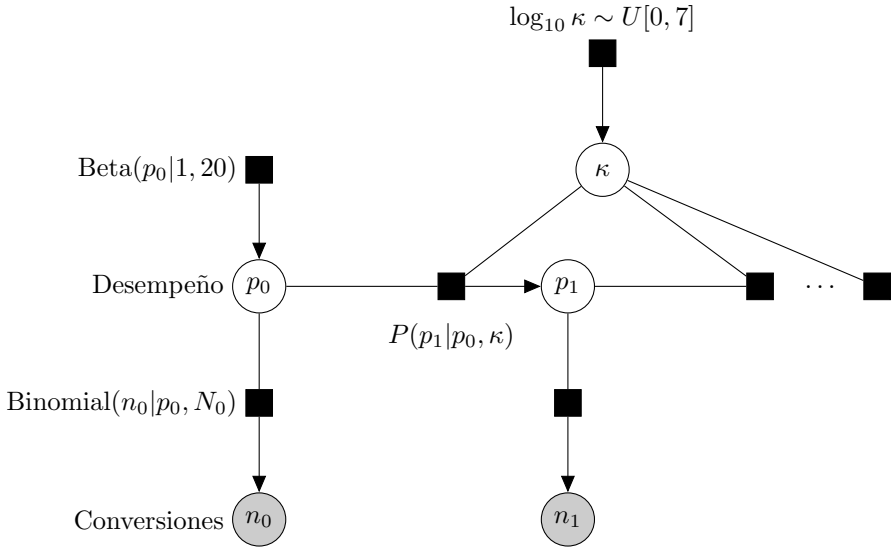


Figura 3: Modelo temporal con cambio gradual del desempeño.

1.1. Aproximación

El modelo temporal de la figura 3 no tiene solución analítica. Para realizar la inferencia se lo puede implementar en cualquier lenguaje de programación probabilística (PPL). Aquí proponemos una aproximación por pasaje de mensajes, que es la que implementamos en clase.

En cada paso temporal, en lugar de aplicar el kernel, se descuentan los conteos que llegan de los meses vecinos, conservando su media,

$$\text{posterior}(p_t) = \text{Beta}\big(p_t \,|\, \lambda\,\alpha_f + \lambda\,\alpha_b + \alpha_l\,,\ \lambda\,\beta_f + \lambda\,\beta_b + \beta_l\big) \quad \text{con} \ \ 0 < \lambda \leq 1 \tag{4}$$

donde el subíndice f, b y l representa los parámetros de los mensajes de *forwardPosterior*(p_{t+1}), *backwardPosterior*(p_{t+1}) y *likelihood*(p_t) respectivamente. La figura 4 muestra estos mensajes que recibe y envía la variable p_t.

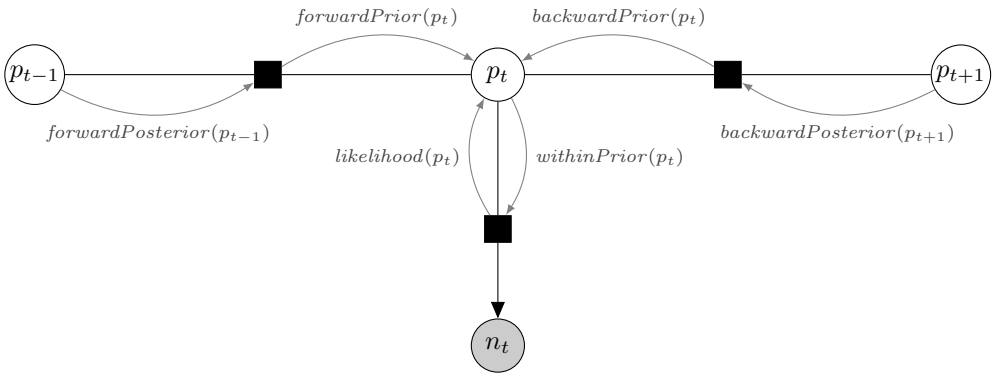


Figura 4: Mensajes que recibe y envía la variable p_t.

Cada mensaje se representa con un par de conteos, que son los parámetros de su Beta. Multiplicar mensajes equivale a sumar sus conteos, y el descuento equivale a multiplicarlos por λ.

$$\begin{aligned}
\textit{likelihood}(p_t) &= (n_t,\ N_t - n_t) \\
\textit{forwardPrior}(p_t) &= \lambda \cdot \textit{forwardPosterior}(p_{t-1}) \quad , \quad \textit{forwardPrior}(p_0) = (1, 20) \\
\textit{backwardPrior}(p_t) &= \lambda \cdot \textit{backwardPosterior}(p_{t+1}) \quad , \quad \textit{backwardPrior}(p_T) = (0, 0) \\
\textit{forwardPosterior}(p_t) &= \textit{forwardPrior}(p_t) + \textit{likelihood}(p_t) \\
\textit{backwardPosterior}(p_t) &= \textit{backwardPrior}(p_t) + \textit{likelihood}(p_t) \\
\textit{withinPrior}(p_t) &= \textit{forwardPrior}(p_t) + \textit{backwardPrior}(p_t) \\
\text{posterior}(p_t) &= \textit{withinPrior}(p_t) + \textit{likelihood}(p_t)
\end{aligned} \tag{5}$$

Los mensajes que p_t envía a un vecino no incluyen la información que recibió de ese mismo vecino, para no contar los datos más de una vez. El prior $(1, 20)$ entra una única vez, por el extremo forward, y el mensaje backward del último mes es neutro. Alcanza con una pasada forward y una pasada backward. El *withinPrior* no contiene los datos del mes t. Para elegir λ buscamos maximizar la evidencia que obtenemos con el *withinPrior*,

$$P(n_t|N_t, \lambda) = \text{BetaBinomial}(n_t|N_t, \textit{withinPrior}(p_t)) \tag{6}$$

que al recibir la información de toda la red (forward y backward) es como un leave-one-out. La evidencia de λ es el producto de esta predicción sobre todos los meses, es decir, la suma de sus logaritmos.

Objetivo: implementar el pasaje de mensajes. Lo mínimo es tener en su repositorio su versión del algoritmo que implementamos en clase.

1. Implementar los mensajes de la ecuación anterior, con una pasada forward y una backward, para la cadena de control p_t y para la de tratamiento q_t.
2. Elegir λ maximizando la log-evidencia, por separado para cada cadena, y graficar la log-evidencia en función de λ.
3. Graficar, mes a mes, el posterior de p_t y de q_t con el λ elegido.
4. Calcular, mes a mes, $P(q_t > p_t|\text{Datos})$ y compararla con la que se obtiene analizando cada mes por separado con el modelo base.

Opcional: implementar el modelo temporal de la figura 3 en un PPL y comparar sus posteriors con los del pasaje de mensajes.

2. Experimento con datos escasos y corrompidos.

¿Cuál es el desempeño de test diagnóstico de Chagas? En el sistema de salud también se realizan experimentos para evaluar el desempeño de los test diagnósticos. En este ejercicio vamos a estudiar un experimento realizado por un importante grupo de investigación para estudiar el desempeño de un test diagnóstico ante tres cepas distintas del parásito, DTU1, DTU2, DTU6. Para ello enviaron a 14 laboratorios, para cada cepa, 3 muestras con una concentración baja del parásito (20 par.Eq/ml) y 3 muestras con una concentración alta del parásito (50 par.Eq/ml).

Cada laboratorio tiene a disposición 3 muestras para cada contexto (cepa/concentración). El archivo `02-positivos_por_cepas_verdaderos.csv` reporta la proporción (0, 1/3, 2/3, 1) de diagnósticos positivos que cada laboratorio tuvo para cada contexto. El desempeño del test podría estimarse usando el modelo base Beta-Binomial de la figura 1. En cada contexto C (cepa/concentración) hay una probabilidad oculta p_C de producir un diagnóstico positivo. En base a esa probabilidad se produce la cantidad de diagnósticos positivos en cada contexto, a la que llamamos n_C.

El problema de los experimentos es siempre el mismo: son costosos de hacer y por lo tanto generalmente tenemos pocos datos. La cantidad de datos por contexto es $14 \cdot 3 = 42$. Pero este no es el único problema. En la vida real, los datos llegan contaminados por errores de carga u otros eventos inesperados. En este caso, todo parece indicar que los operadores (empleados de los laboratorios) realizan a veces su trabajo de forma imprecisa, aplicando de forma errónea el protocolo requerido para el diagnóstico, o simplemente cargando los datos con error en la planilla. Esto hace que el análisis del experimento se vuelva casi imposible mediante métodos clásicos.

Para resolver el problema necesitamos proponer un modelo causal que represente lo más fielmente posible la realidad causal subyacente. Es importante que el modelo que propongamos considere la posibilidad de que los operadores trabajen con error. El objetivo es poder detectar si efectivamente el desempeño del test en las 3 cepas son significativamente diferentes, al menos en el caso en los que la concentración del parásito en la muestra es baja.

Proponemos el siguiente modelo. Cada contexto c (cepa/concentración) tiene un desempeño p_c, que es la probabilidad de que el test dé positivo en una muestra con el parásito. Cada operador i puede trabajar bien ($C_i = 0$) o mal ($C_i = 1$), y trabaja de la misma forma en todas sus muestras. Cuando un operador trabaja mal, la probabilidad de obtener un positivo se reduce por un factor r.

$$\begin{aligned} p_c &\sim \text{Beta}(2, 2) \\ q &\sim \text{Beta}(1, 9) \\ r &\sim \text{Beta}(1, 1) \\ C_i &\sim \text{Bernoulli}(q) \\ n_{c,i} &\sim \begin{cases} \text{Binomial}(3,\, p_c) & \text{si } C_i = 0 \\ \text{Binomial}(3,\, p_c\, r) & \text{si } C_i = 1 \end{cases} \end{aligned} \tag{7}$$

donde $n_{c,i}$ es la cantidad de diagnósticos positivos que obtuvo el operador i en el contexto c, q es la probabilidad de que un operador trabaje mal y r, un número entre 0 y 1, es el factor de

reducción. El prior Beta$(1, 9)$ expresa que esperamos que la mayoría de los operadores trabajen bien. Como cada operador tiene un único C_i para todos los contextos, un operador que obtiene pocos positivos en varios contextos se vuelve sospechoso en todos ellos. Con los datos de este experimento, el modelo con operadores tiene una log-evidencia alrededor de 6,5 mayor que la del modelo base, es decir, es del orden de 600 veces más probable.

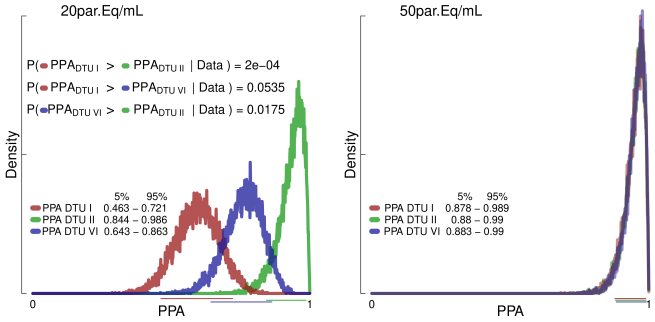


Figura 5: Resultado del grupo de investigación

2.1. Cómo comparar modelos

El archivo `comparacion_modelos.py` implementa en PyMC los dos modelos y los compara. Lee los datos de `02-positivos_por_cepas_verdaderos.csv` y los guarda en la matriz `n`, que tiene la cantidad de positivos (de 0 a 3) de cada operador (fila) en cada contexto (columna). Cada modelo es una función de Python que recibe el vector `p` con el desempeño de los seis contextos, $p_c \sim \text{Beta}(2, 2)$, compartido por todos los modelos. Dentro de la función se crean los parámetros propios del modelo y se devuelve la log-verosimilitud de todos los datos, $\log P(\text{Datos}|\theta, M)$. Las funciones `base` y `operadores` sirven de ejemplo.

En el modelo con operadores, la variable C_i no se crea explícitamente, sino que se suma sobre sus dos valores posibles, $C_i = 0$ y $C_i = 1$,

$$P(n_{1,i}, \ldots, n_{6,i}|p, q, r) = (1 - q) \prod_c \text{Binomial}(n_{c,i}|3, p_c) + q \prod_c \text{Binomial}(n_{c,i}|3, p_c\, r) \qquad (8)$$

Sumar sobre las variables discretas facilita el muestreo, porque NUTS solo trabaja con variables continuas. Como la función devuelve logaritmos, la suma de los dos términos se calcula con `pt.logaddexp`, que hace $\log(e^a + e^b)$ sin perder precisión. Si su modelo tiene otras variables discretas por operador, conviene sumarlas de la misma forma.

Para agregar un modelo hay que escribir una nueva función y sumarla al diccionario `modelos`,

```
def mi_modelo(p):
    s = pm.Beta("s", alpha=1, beta=1) # parametros propios del modelo
    return pm.logp(pm.Binomial.dist(n=N, p=p * s), n).sum()

modelos = {"Base": base, "Operadores": operadores,
           "Mi modelo": mi_modelo}
```

Los nombres de los parámetros tienen que ser distintos a los de los otros modelos (por ejemplo, no se puede volver a usar `q` o `r`). Si el modelo no usa el vector `p`, alcanza con ignorarlo. El archivo se ejecuta con `python comparacion_modelos.py`.

Para comparar los modelos, el archivo agrega una variable discreta M, el selector, que elige cuál de los modelos genera los datos. Con un prior uniforme sobre M, la regla de Bayes dice que $P(M|\text{Datos}) \propto P(\text{Datos}|M)$, es decir, la probabilidad a posteriori de cada modelo es proporcional a su evidencia. El archivo imprime, para cada modelo, cuántas veces lo visitó el selector, su probabilidad a posteriori $P(M|\text{Datos})$ y su log-evidencia relativa a la del mejor modelo. Si un modelo es mucho peor que otro, el selector lo visita pocas veces y su evidencia queda estimada con imprecisión; si no lo visita nunca, solo sabemos que su probabilidad a posteriori es menor a

uno sobre la cantidad total de muestras. Si PyMC advierte divergencias o valores de $\hat{r}$ mayores a 1.01, hay que aumentar la cantidad de muestras (`muestras`).

El selector sirve para comparar modelos, pero no para estudiar los posteriors de un modelo. Para eso hay que ajustar el modelo elegido solo, sin el selector, agregando al final del archivo,

```
with pm.Model():
    p = pm.Beta("p", alpha=2, beta=2, shape=len(contextos))
    pm.Potential("verosimilitud", operadores(p))
    traza = pm.sample(2000, tune=2000, chains=4, target_accept=0.95)
```

La traza tiene muestras conjuntas de p y de los parámetros propios del modelo. Como C_i fue sumada, su posterior se recupera a partir de esas muestras. Para cada muestra (p, q, r) se calcula la probabilidad de que el operador i haya trabajado mal,

$$P(C_i = 1|n_i, p, q, r) = \frac{q \prod_c \text{Binomial}(n_{c,i}|3, p_c\, r)}{(1-q) \prod_c \text{Binomial}(n_{c,i}|3, p_c) + q \prod_c \text{Binomial}(n_{c,i}|3, p_c\, r)} \tag{9}$$

y el promedio sobre todas las muestras es $P(C_i = 1|\text{Datos})$.

2.2. Modelo causal alternativo

El objetivo es proponer un modelo causal alternativo que tenga mayor evidencia que el modelo con operadores. No es requisito encontrar un modelo con mayor evidencia. Lo que va a ser evaluado es el argumento en lenguaje natural que sostiene el modelo y, si lo implementan, la correcta comparación de modelos. Algunas preguntas que pueden servir de guía: ¿Es razonable que todos los operadores que trabajan mal lo hagan en la misma medida? ¿Hay alguna relación causal entre el desempeño del test en concentración baja y en concentración alta?

Objetivo: proponer un modelo causal alternativo. Lo mínimo es pensar el modelo y especificarlo en papel.

1. Especificar matemáticamente el modelo propuesto, con ecuaciones o con su grafo de factores, y justificar en lenguaje natural sus supuestos causales.

Objetivo opcional (muy recomendado): implementar y comparar el modelo.

1. Implementarlo en el archivo `comparacion_modelos.py` (ver sección 2.1) y reportar la log-evidencia relativa y la probabilidad a posteriori de los tres modelos: Base, Operadores y el propuesto.
2. Verificar que la inferencia haya convergido y reportar cuántas veces el selector visitó cada modelo.
3. Con el modelo de mayor evidencia:
   a) Graficar el posterior de la probabilidad de que cada operador haya trabajado mal.
   b) Graficar el posterior marginal del desempeño del test en cada contexto, en concentración baja y alta.
   c) Calcular de a pares la probabilidad de que el desempeño del test en una cepa sea superior al de otra en concentración baja, $P(p_i > p_j|\text{Datos})$. Esta probabilidad debe calcularse con las muestras conjuntas del posterior y no con el producto de las marginales, porque en estos modelos los desempeños están correlacionados a través de los parámetros de los operadores.

3. Evaluación de tests diagnósticos sin gold standard.

3.1. Introducción

La evaluación del desempeño de un test diagnóstico es sencilla cuando disponemos de datos "gold standard", que nos garantizan el valor real de la clase. En ese caso, podemos calcular la probabilidad de diagnóstico dado el estado real a través de la tabla 1 de contingencias.

	D = Positivo = 1	D = Negativo = 0
Estado = Verdadero	**Sensibilidad**, s	1-Sensibilidad, $1 - s$
Estado = Falso	1-Especificidad, $1 - x$	**Especificidad**, x

Cuadro 1: P(Diagnóstico|Estado)

Sin embargo todos los test son imperfectos en alguna medida, por lo cual en general no podemos observar directamente esta tabla. En estos casos es necesario inferir las *variables ocultas* a partir de los diagnósticos reportados por los diferentes test: el estado real de las personas, la sensibilidad y especificidad de los test y la prevalencia de la enfermedad en la población.

Los modelos bayesianos de evaluación de test diagnóstico se los conoce en la literatura médica y veterinaria como Bayesian Latent Class Analysis [1, 2]. Dos modelos B-LCA que tienen gran aceptación en la comunidad de evaluación de test diagnóstico son el modelo Hui-Walter (1980) y el modelo Dendukuri (2001). El modelo Hui-Walter, en su versión bayesiana, es capaz de estimar los desempeños correctamente (y con baja incertidumbre) cuando los test son independientes entre sí. Cuando existen dependencias entre tests, sin embargo, el modelo Hui-Walter puede sobreestimar los desempeños de los tests. En los años 2000 se proponen los primeros modelos de clases latentes que incorporan una variable adicional para estimar la correlación entre pares de test [3,4]. Como desventaja, este modelo solo permite evaluar el desempeño de a pares de tests, produciendo estimaciones con mucha incertidumbre.

Entre 2019 y 2024, la European Cooperation in Science and Technology financió la red "Harmony" (COST Action CA18208)[1], cuyo objetivo fue promover el uso de los modelos B-LCA en estadística, epidemiología y salud poblacional. El segundo taller de Harmony sobre evaluación de test diagnóstico (Copenhague, 2021)[2] lo dictó Matthew Denwood, autor del paquete `runjags` de R[3]. La función `template_huiwalter()` de ese paquete genera un modelo B-LCA para el lenguaje de programación probabilística JAGS. En la sesión 4 del taller se usó para estimar el desempeño de 3 tests en un escenario simulado en el que 2 de ellos tienen covarianza. Como vamos a ver, el modelo con covarianzas que genera `template_huiwalter()` tiene un error, tanto en términos teóricos como prácticos.

3.2. Modelo 0: una población y un test (no identificable).

El resultado de un test diagnóstico en N pacientes puede representarse como un vector $r = (n_0, n_1)$, donde n_0 y n_1 son la cantidad de diagnósticos negativos y positivos respectivamente, con $N = n_0 + n_1$. Los modelos suponen que la probabilidad de diagnósticos negativo y positivo, $\boldsymbol{q} = (q_0, q_1)$, depende de la sensibilidad s y especificidad x del test, pero también de la prevalencia de la enfermedad en la población, $q_1 = ps+(1-p)(1-x)$. Luego, la probabilidad del resultado observado es $P(r|q, N) = \text{Binomial}(r|q, N)$. Finalmente, la prevalencia, la sensibilidad y la especificidad se modelen mediante distribuciones Beta$(\cdot, \cdot)$.

Cualquier modelo probabilístico puede especificarse usando métodos gráficos, que son de gran ayuda para la lectura y la intuición. Aquí expresaremos los modelos usando *factor graphs*. En particular, los *factor graph* son grafos bipartitos, compuestos por nodos "variables" (p, s, x, q)

[1] `https://www.cost.eu/actions/CA18208/`

[2] Materiales en `https://harmony-net.eu/denmark-training-school-material/` y código en `https://github.com/ku-awdc/HarmonyWS2`.

[3] `https://cran.r-project.org/web/packages/runjags/index.html`

y por nodos "funciones" (distribuciones de probabilidad). Los ejes representan la relación "la variable v es parámetro de la función f". La dirección de los ejes indican el sentido de la relación causal.

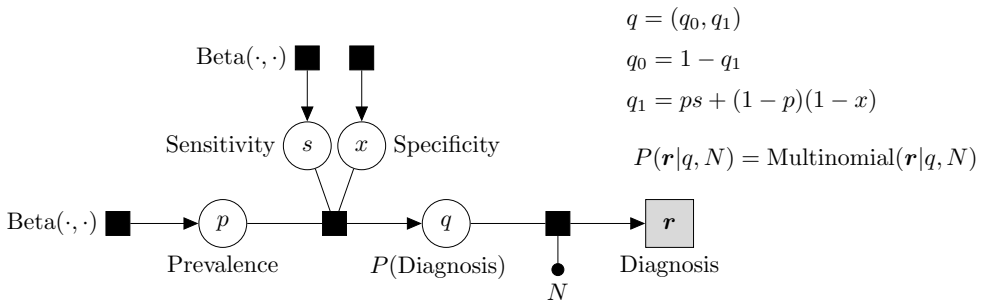


Figura 6: Modelo de una población y un test.

Las variables ocultas se dibujan en blanco y las observables en gris. Los círculos y los cuadrados representan variables continuas y discretas respectivamente. Los puntos negros representan constantes.

Este modelo tiene 3 grados de libertad (p, s y x) y un solo dato $n_0 = N - n_1$. Esto quiere decir que hay muchas combinaciones de p, s y x que son igualmente verosímiles y por lo tanto indistinguibles entre sí. Para poder identificar el valor real es necesario tener al menos la misma cantidad de datos que de hipótesis (mismas variables libres que variables restrictoras).

Este modelo básico no es capaz de identificar (o estimar) las hipótesis debido a que hay más variables libres que datos.

3.3. Modelo 1 "Hui-Walter": varias poblaciones y tests.

El Modelo 1 (Hui-Walter) es igual al anterior, pero con al menos dos test y dos comunidades. El resultado observable de dos test es un vector, $\boldsymbol{r} = (n_0, n_1, n_2, n_3)$, donde n_0, n_1, n_2, y n_3 representa la cantidad de resultados que son conjuntamente 00, 01, 10 y 11 respectivamente. El modelo Hui-Walter supone que la sensibilidad y especificidad de los test (s_t y x_t) son independientes entre sí. Luego, la probabilidad del resultado conjunto es un vector $\boldsymbol{q} = (q_0, q_1, q_2, q_3)$, donde $q_0 = p\,(1 - s_a)\,(1 - s_b) + (1 - p)\,x_a\,x_b$ (ver el resto de las probabilidades q_i en la figura 7). Y con N pacientes la probabilidad es $P(r|q, N) = \text{Multinomial}(r|q, N)$.

La especificación gráfica del modelo Hui-Walter es similar al modelo anterior, salvo que las estructuras que aparecen repetidas veces se representan de forma plegada usando placas que tienen por subíndice la variable que despliega la repetición, aquí por test y por comunidades.

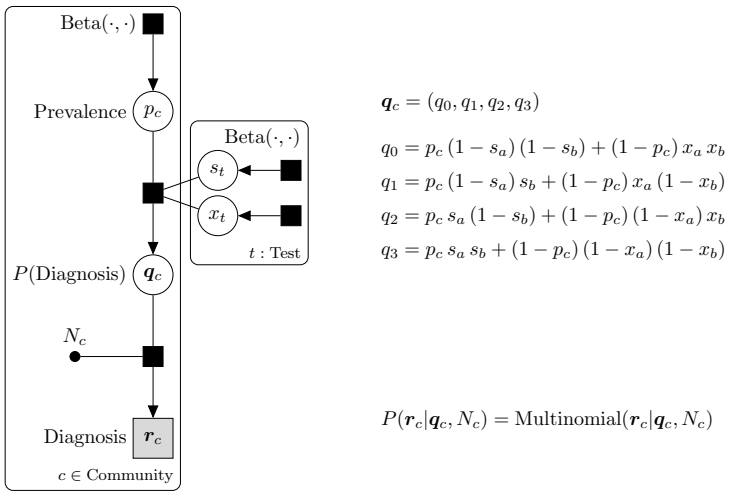


Figura 7: Modelo Hui-Walter [5]

Con dos poblaciones y dos test el modelo ya puede estimar las 6 hipótesis (p_c, s_t, x_t) gracias a que tenemos 6 datos (3 por cada $\boldsymbol{r}_c$).

3.4. Modelo 2 "Dendukuri": con una covarianza.

Para evaluar la sensibilidad y especificidad en presencia de dependencias condicionales se han propuesto modelos alternativos [3, 6, 7], en los cuales se incorporan variables que estiman la covarianza entre tests. Describiremos aquí el modelo de efectos fijos de la sección 2 del paper de Dendukuri [3]. En el siguiente modelo gráfico agregamos dos variables c_s y c_x que representan la covarianza de la sensibilidad y especificidad entre un par de test.

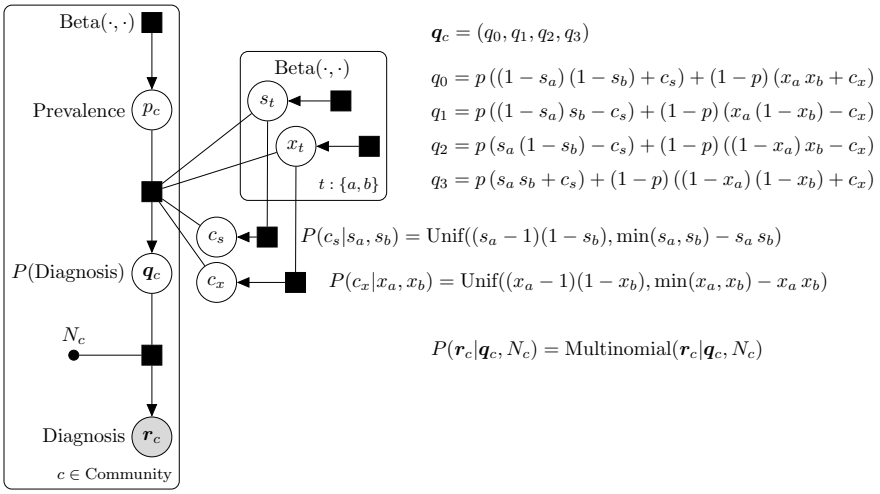


Figura 8: Modelo Dendukuri "fixed effect model" con covarianzas (2001)

Recordamos primero la definición de covarianza.

$$\begin{aligned}
\text{cov}(x, y) &= \mathbb{E}[(X - \mu_x)(Y - \mu_y)] \\
&= \mathbb{E}[X\,Y - \mu_x\,Y - X\,\mu_y + \mu_x\,\mu_y] \\
&= \mathbb{E}[X\,Y] - \mu_x\mathbb{E}[Y] - \mathbb{E}[X]\mu_y + \mu_x\,\mu_y] \\
&= \mathbb{E}[X\,Y] - \mu_x\,\mu_y - \cancel{\mu_x\,\mu_y} + \cancel{\mu_x\,\mu_y}] \\
&= \mathbb{E}[X\,Y] - \mathbb{E}[X]\,\mathbb{E}[Y]
\end{aligned}$$

donde μ es la media de las variables. Notar que la covarianza está definida sobre un par de variables, $\text{cov}(x, y)$. En el caso especial de variables binarias ($x, y \in \{0, 1\}$) la esperanza del producto es igual a la probabilidad conjunta cuando las dos variables son positivas,

$$\mathbb{E}[XY] = \sum_{\substack{x \in X \\ y \in Y}} P(x, y)\,x\,y = P(x = 1, y = 1)$$

Y de forma similar, la esperanza de una variable es igual a la probabilidad de que esa variable sea positiva,

$$\mathbb{E}[X] = \sum_{x \in X} P(x)\,x = P(x = 1)$$

Luego, esto es lo que nos permite escribir la covarianza en términos de la probabilidades conjunta y marginal de ambas variables positivas,

$$\begin{aligned}
\text{cov}(X, Y) &= P(x = 1, y = 1) - P(x = 1)P(y = 1) \\
P(x = 1, y = 1) &= P(x = 1)P(y = 1) + \text{cov}(X, Y)
\end{aligned} \tag{10}$$

Esta propiedad nos permite recuperar toda la distribución conjunta, $P(x, y)$, conociendo solamente las probabilidades marginales de ambas variables positivas, $P(x = 1)$ y $P(y = 1)$, y su covarianza $\text{cov}(x, y)$. La tabla 2 representa la distribución conjunta de 2 variables binarias con las variables

$P(x, y)$	$x = 0$	$x = 1$	
$y = 0$			$1 - P(y = 1)$
$y = 1$		$P(x = 1)P(y = 1) + \text{cov}(x, y)$	$P(y = 1)$
	$1 - P(x = 1)$	$P(x = 1)$	1

Cuadro 2: Reconstrucción de la distribución conjunta a partir de las marginales y la covarianza.

latentes que queremos estimar: marginales y covarianza. Dada la información de las marginales y la covarianza podemos reconstruir la distribución conjunta.

$$\begin{aligned}
P(x = 1, y = 1) &= P(x = 1)P(y = 1) + \text{cov}(x, y) \\
P(x = 0, y = 1) &= P(y = 1) - P(x = 1, y = 1) \\
&= P(y = 1) - P(x = 1)P(y = 1) - \text{cov}(x, y) \\
&= P(y = 1)(1 - P(x = 1)) - \text{cov}(x, y) \\
P(x = 0, y = 0) &= (1 - P(x = 1)) - P(x = 0, y = 1) \\
&= (1 - P(x = 1))(1 - P(y = 1)) + \text{cov}(x, y) \\
P(x = 1, y = 0) &= P(x = 1) - P(x = 1, y = 1) \\
&= P(x = 1)(1 - P(y = 1)) - \text{cov}(x, y)
\end{aligned}$$

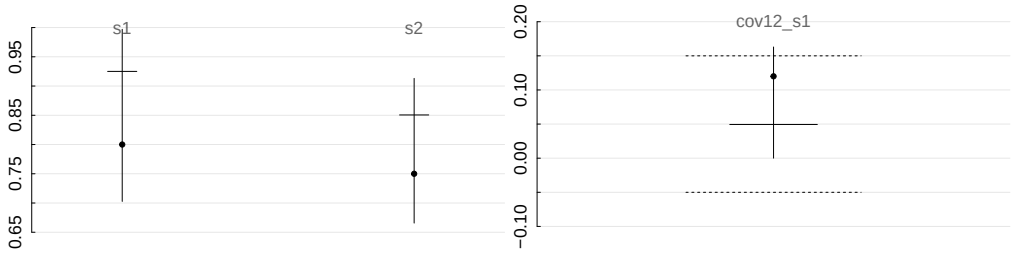
El resultado final de estas cuatro ecuaciones es lo que se utiliza en el modelo Dendukuri para definir la variable $\boldsymbol{q}_c$ en la figura 8. Notar que en el modelo de Dendukuri hay dos distribuciones conjuntas independientes: la que corresponde con la sensibilidad $P(\text{diagnóstico}_a, \text{diagnóstico}_b|\text{estado} = 1)$, y la que corresponde con la especificidad $P(\text{diagnóstico}_a, \text{diagnóstico}_b|\text{estado} = 0)$.

3.4.1. Evaluación del modelo Dendukuri

Para evaluar el modelo vamos a simular datos a partir del siguiente ejemplo.

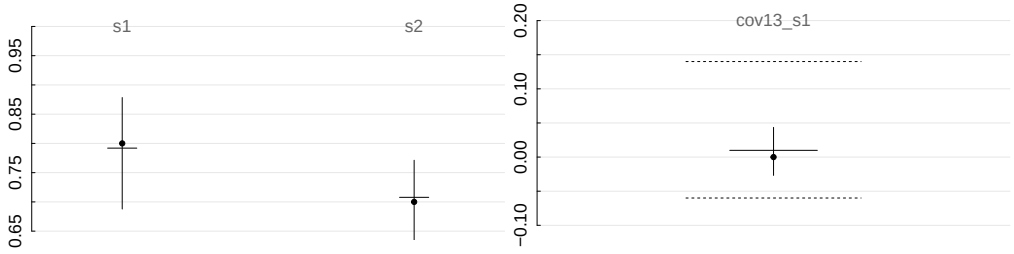
$$P(x = 1) = 0{,}80$$
$$P(y = 1) = 0{,}75$$
$$\text{cov}(x, y) = 0{,}12$$

Si estudiamos la covarianza de a pares de test mediante el modelo de Dendukuri, realizando la inferencia con el paquete `runjags`, obtenemos los siguientes resultados.



(a) Sensibilidad (b) Covarianza

Figura 9: Valores verdaderos (puntos) e intervalos de credibilidad (rangos) obtenidos con el modelo Dendukuri para un par de tests con covarianza positiva (cov=0.12).



(a) Sensibilidad (b) Covarianza

Figura 10: Valores verdaderos (puntos) e intervalos de credibilidad (rangos) obtenidos con el modelo Dendukuri para un par de tests sin covarianza (cov=0).

En todos los casos las estimaciones contienen los verdaderos valores.

3.5. Modelo 3 "Harmony": con varias covarianzas.

El modelo Dendukuri (2001) ha sido adoptado ampliamente en la literatura de evaluación bayesiana de test diagnóstico. Como desventaja, este modelo ha sido definido para evaluar únicamente tests de a pares. ¿Es posible extender el modelo Dendukuri 2001 para estimar correctamente el desempeño de los tests diagnóstico en contextos de dependencia entre varios tests?

La función `template_huiwalter()` del paquete `runjags` propone una extensión para más de dos tests. En el siguiente código mostramos la primera parte del modelo mencionado.

```
for(p in 1:Populations){

      # Probability of observing Test1- Test2- Test3- from a true positive::
      se_prob[1,p] <- prev[p] * ((1-se[1])*(1-se[2])*(1-se[3]) +covse12 +covse13 +covse23)
      # Probability of observing Test1- Test2- Test3- from a true negative::
      sp_prob[1,p] <- (1-prev[p]) * (sp[1]*sp[2]*sp[3] +covsp12 +covsp13 +covsp23)
```

```
# Probability of observing Test1+ Test2- Test3- from a true positive::
se_prob[2,p] <- prev[p] * (se[1]*(1-se[2])*(1-se[3]) -covse12 -covse13 +covse23)
# Probability of observing Test1+ Test2- Test3- from a true negative::
sp_prob[2,p] <- (1-prev[p]) * ((1-sp[1])*sp[2]*sp[3] -covsp12 -covsp13 +covsp23)
```

En términos generales, el modelo propuesto implementado en la función `template_huiwalter()` define la distribución de probabilidad conjunta de dimensionalidad n, $P(\wedge_{i=1}^{n} X_i = v_i)$, función de las marginales, $P(X_i = v_i)$, y las covarianzas $\text{cov}(X_i, Y_i)$.

$$P(\wedge_{i=1}^{n} X_i = v_i) = \left(\prod_{i=1}^{n} P(X_i \neq v_i) \right) + \sum_{i=1}^{n-1} \sum_{i+1}^{n} (-1)^{\mathbb{I}(v_i = v_j)} \text{cov}(X_i, Y_i) \tag{11}$$

donde $\mathbb{I}(v_i = v_j)$ representa la función indicadora que vale 1 si la se cumple la igualdad y 0 en caso contrario. Notar la diferencia de este modelo de covarianzas respecto del modelo descripto en la sección anterior. En la sección anterior derivamos el modelo de covarianza para el caso de dos variables binarias. Nos preguntamos ahora si esta propuesta de extensión del modelo de Dendukuri es teóricamente correcta y empíricamente efectiva.

3.5.1. Evaluación teórica del modelo Harmony

El principal problema teórico del modelo propuesto está relacionado con el hecho de que no es posible determinar los valores de las distribuciones de probabilidad conjunta de dimensionalidad mayor a 2 teniendo como datos únicamente las probabilidades marginales y las covarianzas. Por ejemplo, la distribución conjunta de tres variables binarias, $P(x, y, z)$, tiene 7 variables libres (8 probabilidades y la restricción de que sumen 1) y solo contamos con 6 datos (las 3 marginales $P(x = 1)$, $P(y = 1)$, $P(z = 1)$ y las 3 covarianzas $\text{cov}(x, y)$, $\text{cov}(x, z)$, $\text{cov}(y, z)$). Esto significa que existen infinitas distribuciones de probabilidad $P(x, y, z)$ compatibles con los datos. Esto puede tener consecuencias negativas a la hora de identificar las variables latentes, pero no implica que el modelo sea teóricamente incorrecto.

Sin embargo, si el modelo propuesto es teóricamente correcto debe ocurrir que la distribución conjunta de dimensionalidad n sea compatible con la información de las marginales y sus covarianzas. Por ejemplo, en el caso de dimensionalidad 3 debe ocurrir $\sum_{k=0}^{1} P(x, y, z = k) = P(x, y)$. Si nos basamos en el modelo "Harmony" de covarianza, debe valer por definición que

$$\begin{aligned} P(x = 1, y = 1) &= P(1, 1, 0) + P(1, 1, 1) \\ &= P_x P_y P_z + C_{xy} + \cancel{C_{xz}} + \cancel{C_{yz}} \\ &+ P_x P_y (1 - P_z) + C_{xy} - \cancel{C_{xz}} - \cancel{C_{yz}} \\ &= P_x P_y (1 - \cancel{P_z} + \cancel{P_z}) + \mathbf{2} C_{xy} \end{aligned} \tag{12}$$

donde usamos los renombres $P_x = P(x = 1)$ y $C_{xy} = \text{cov}(x, y)$. Notar que la solución de esta ecuación 12 tiene una covarianza adicional respecto del verdadero valor $P(x = 1, y = 1)$ que hemos derivado en la ecuación 10. Luego, el modelo propuesto implementado en la función `template_huiwalter()` ni siquiera es compatible con las marginales.

3.5.2. Evaluación empírica del modelo Harmony

En esta sección evaluamos en datos simulados el desempeño de la generalización del modelo Dendukuri implementada en el paquete `runjags`. Para ello elegiremos una de las infinitas distribuciones de probabilidad conjunta $P(x, y, z)$ compatible con las siguientes distribuciones marginales y condicionales.

$$\begin{aligned} &P(x = 1) = 0{,}80 && P(x = 1, y = 1) = P(x = 1) P(y = 1 | x = 1) = 0{,}80 \cdot 0{,}90 \\ &P(y = 1) = 0{,}75 && P(x = 1, z = 1) = P(x = 1) P(z = 1) = 0{,}80 \cdot 0{,}70 \\ &P(z = 1) = 0{,}70 && P(y = 1, z = 1) = P(y = 1) P(z = 1) = 0{,}75 \cdot 0{,}70 \end{aligned}$$

De aquí se desprende que las covarianzas son,

$$\text{cov}(x, y) = P(x = 1, y = 1) - P(x = 1)P(y = 1) = 0{,}72 - 0{,}60 = 0{,}12$$
$$\text{cov}(x, z) = \text{cov}(y, z) = 0$$

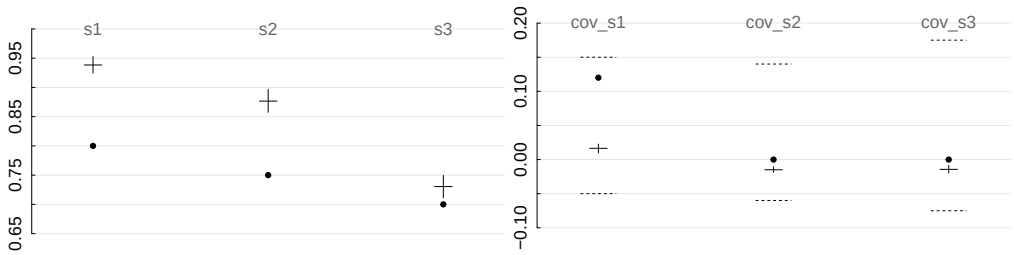
Esta información determina infinitas distribuciones conjuntas de a 3 variables ($P(v_1, v_2, v_3)$). Para la simulación de los datos tomamos la siguiente distribución conjunta.

P_{xy0}	x0	x1
y0	0.045	0.03
y1	0.015	0.21

P_{xy1}	x0	x1
y0	0.125	0.05
y1	0.015	0.51

Cuadro 3: Distribución conjunta de los diagnósticos usada para simular los datos.

Generamos 3000 pacientes en 4 poblaciones con diversas prevalencias, y generamos los resultados de los test diagnósticos mediante la distribución conjunta 3. Creamos el modelo pasándole los datos simulados a la función `template_huiwalter()` del paquete `runjags` y lo modificamos a mano para activar la estimación de covarianzas. En la siguiente figura se puede observar el resultado de la inferencia obtenida con el paquete `runjags`.



(a) Sensibilidad (b) Covarianza

Figura 11: Valores verdaderos (puntos) e intervalos de credibilidad (rangos) obtenidos con el modelo "Harmony".

Se puede observar que las estimaciones del modelo Harmony no contienen los verdaderos valores con los que se generaron los datos simulados. Es decir, no solo hay un problema teórico con el modelo Harmony. El problema también es práctico.

3.6. Modelo 4: Dendukuri para todos los pares.

En esta sección proponemos una generalización del modelo Dendukuri para T test diagnósticos en total. La solución consiste en evaluar todos los pares de test diagnóstico al mismo tiempo, en un único modelo. Para todo par de test $1 \leq a < b \leq T$, la observación relativa a ese par es $\boldsymbol{r}_{ab} = (n_{ab0}, n_{ab1}, n_{ab2}, n_{ab3})$, el vector de conteos de los diagnósticos conjuntos 00, 01, 10, 11 del par ab. La prevalencia es una variable que depende solo de la comunidad en la que estamos.

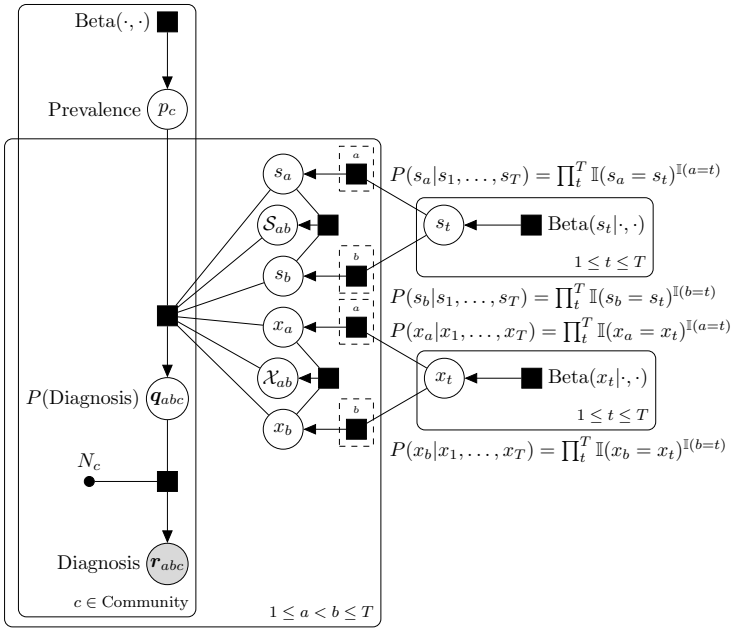


Figura 12: Modelo Dendukuri extendido a todos los pares de tests.

3.6.1. Evaluación en simulaciones

Validamos el modelo de la figura 12 con datos simulados, en los que se conoce el valor verdadero de todos los parámetros. Simulamos dos escenarios con $T = 10$ tests (45 pares), 5 comunidades con prevalencias 0,1, 0,3, 0,5, 0,7, 0,9 y $N = 3000$ individuos por comunidad. Los resultados binarios correlacionados se generan con una cópula gaussiana (método de Emrich y Piedmonte, 1991). Cada individuo tiene un vector de 10 variables normales correlacionadas, y cada test da positivo cuando su variable supera un umbral, elegido para que la probabilidad de positivo sea la sensibilidad (o uno menos la especificidad). Esto garantiza que las covarianzas verdaderas sean compatibles con una distribución conjunta genuina, a diferencia de elegir covarianzas "a mano". Los dos escenarios comparten las mismas sensibilidades y especificidades verdaderas y solo difieren en la estructura de dependencia entre tests:

- `random`: una matriz de correlación aleatoria (pero válida, semidefinida positiva) entre los primeros 7 tests. Los tests 8, 9 y 10 son independientes entre sí y del resto, para chequear que el modelo no "invente" covarianza donde no la hay.
- `two_groups`: dos grupos de 5 tests cada uno, con correlación 0,5 dentro de cada grupo y 0 entre grupos.

En ambos casos la corrida de NUTS (4 cadenas, 1500 muestras de warmup y 2000 muestras) no tuvo divergencias y el peor $\hat{r}$ fue 1,03. Las figuras 13 y 14 muestran, para cada parámetro, el valor verdadero contra la mediana del posterior y su intervalo de credibilidad del 95 % (la diagonal marca la identidad).

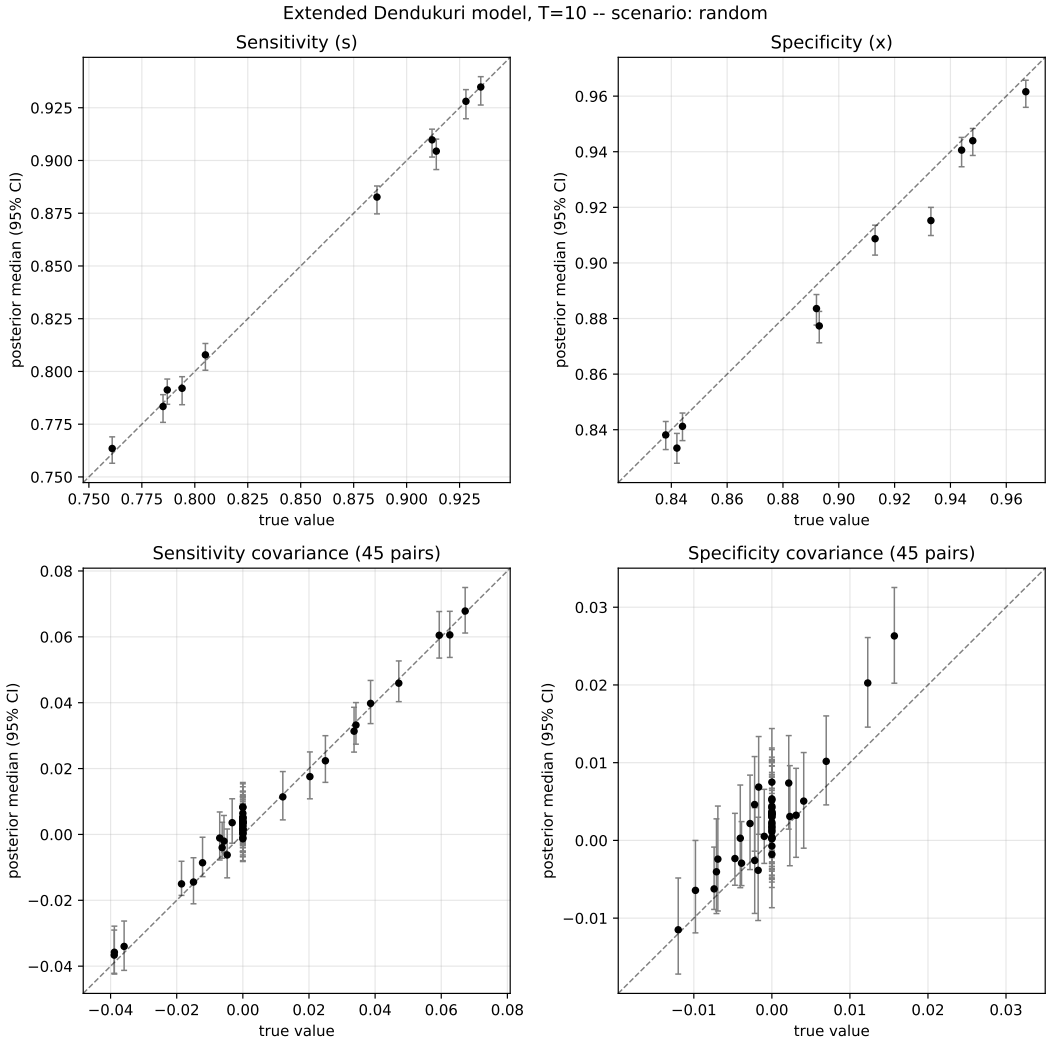


Figura 13: Escenario `random` ($T = 10$): sensibilidad, especificidad y las 45 covarianzas de a pares, valor verdadero contra mediana del posterior e intervalo del 95 %.

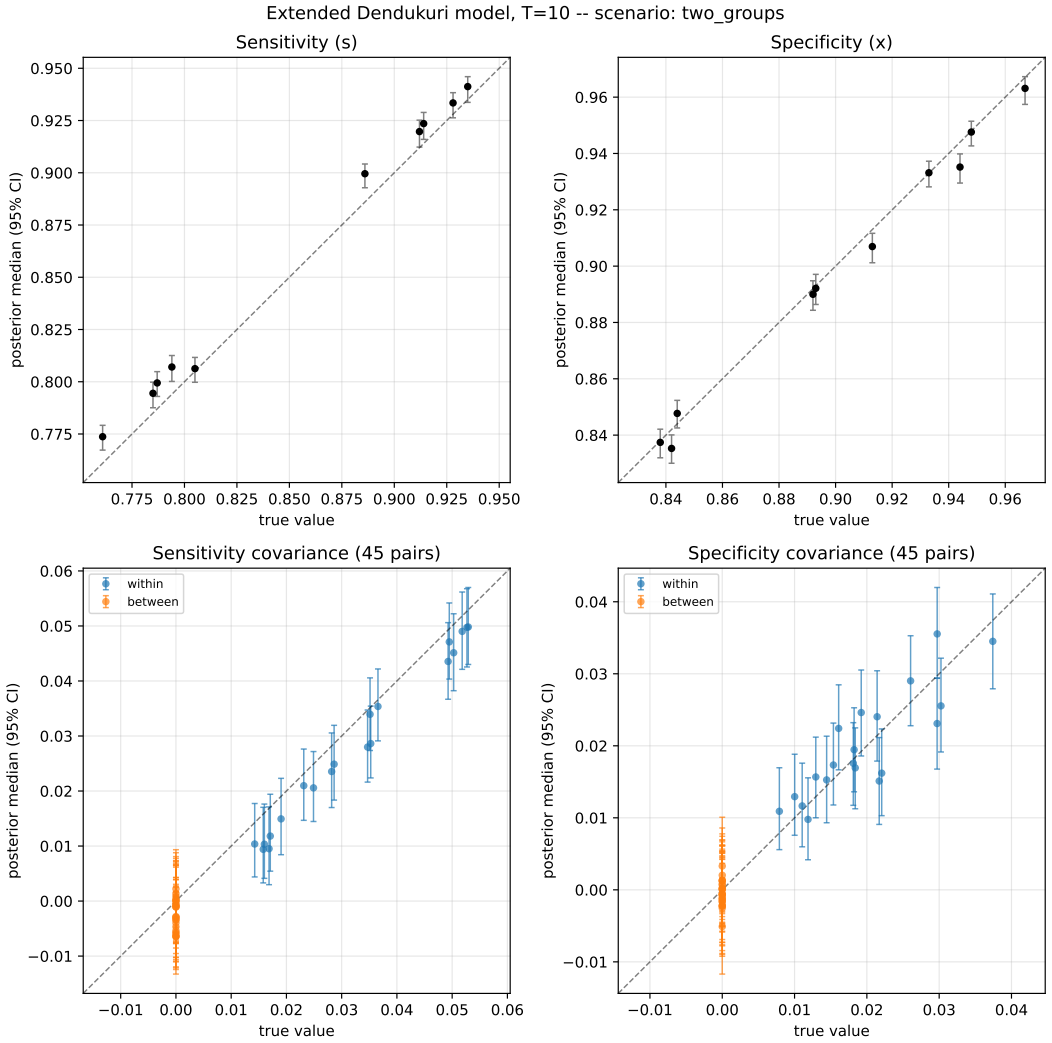


Figura 14: Escenario `two_groups` ($T = 10$, dos grupos de 5 tests): sensibilidad, especificidad y las 45 covarianzas de a pares (coloreadas según si el par es del mismo grupo o de grupos distintos), valor verdadero contra mediana del posterior e intervalo del 95 %.

En el escenario `random`, los 27 pares que involucran a los tests 8, 9 o 10 recuperan una covarianza concentrada muy cerca de 0, consistente con la independencia impuesta. Sobre el total de los 45 pares, el intervalo del 95 % contiene el valor verdadero en el 87 % de las covarianzas de sensibilidad, en el 89 % de las de especificidad y en el 90 % de las sensibilidades. La cobertura de las especificidades fue más baja (50 %), pero se calcula sobre apenas 10 parámetros con intervalos muy angostos (por el N grande de cada comunidad) en una única simulación, así que es un chequeo ruidoso. En el escenario `two_groups` la cobertura de las covarianzas fue casi perfecta (100 % en sensibilidad y 93 % en especificidad), porque la estructura de dos bloques es más fácil de identificar que una matriz de correlación arbitraria. La cobertura de las sensibilidades fue baja (30 %), por la misma razón de los intervalos angostos.

En conjunto, ambos escenarios muestran que el modelo recupera la dirección y la magnitud relativa de las covarianzas verdaderas. En particular, distingue los pares independientes de los correlacionados, sin falsos positivos sistemáticos.

3.7. Datos simulados

Les damos los datos simulados de esta sección y el código que los genera.

El archivo `simulate.py` genera el ejemplo de 3 tests de la sección 3.5.2: 4 comunidades con prevalencias 0,2, 0,4, 0,6 y 0,8, 3000 pacientes por comunidad y la distribución conjunta de la tabla 3. Guarda en `data/covariance3.npz` los conteos de cada par de tests en cada comunidad (`pair_counts`, en el orden 00, 01, 10, 11, para los pares de `pair_keys`) y los valores verdaderos (`prevalences`, `true_s`, `true_x` y `true_cov`). Los modelos Hui-Walter y Harmony con 3 tests necesitan los conteos de los 8 patrones conjuntos, que devuelve la función `simulate_joint_counts` del mismo archivo (el bit t del índice del patrón es el resultado del test $t + 1$).

El archivo `simulate_scenarios.py` genera los escenarios `random` y `two_groups` de 10 tests, con la cópula gaussiana de `copula.py` y las definiciones de `scenarios.py`. Guarda en `data/scenario_random.npz` y `data/scenario_two_groups.npz` los conteos de a pares (`pair_keys` y `pair_counts`), los valores verdaderos (`true_p`, `true_s`, `true_x`, `true_cov_s` y `true_cov_x`) y los conteos de los patrones completos de cada comunidad (`obs_community`, `obs_bits` y `obs_count`). Los archivos se leen con `numpy.load`.

Objetivo: probar los modelos descriptos. Lo mínimo es implementar los cuatro modelos en el lenguaje de programación probabilística que prefieran y evaluarlos contra los valores verdaderos.

1. Con los datos de 3 tests, implementar el modelo Hui-Walter y comparar las sensibilidades y especificidades estimadas con las verdaderas. ¿Qué pasa con los tests 1 y 2, que tienen covarianza?
2. Con los mismos datos, implementar el modelo Dendukuri para los pares de tests 1-2 y 1-3, y comparar sus estimaciones con los valores verdaderos, como en las figuras 9 y 10.
3. Implementar el modelo Harmony con los 3 tests y mostrar que sus estimaciones no contienen los valores verdaderos, como en la figura 11. Explicar el resultado a partir de la ecuación 12.
4. Implementar el modelo Dendukuri para todos los pares y estimar las sensibilidades, especificidades y covarianzas en los escenarios `random` y `two_groups`. Graficar el valor verdadero contra el posterior, como en las figuras 13 y 14.
5. Verificar en cada caso que la inferencia haya convergido (sin divergencias y con $\hat{r}$ menor a 1.01).

Objetivo opcional: un modelo que mejore todo lo anterior. Ninguno de los modelos anteriores es el que genera los datos de los escenarios de 10 tests. Proponer e implementar un modelo que estime mejor los valores verdaderos que el modelo Dendukuri para todos los pares. Hay al menos dos caminos.

1. El modelo que genera los datos: cada individuo tiene un estado oculto y, dado el estado, los resultados de los tests son variables normales correlacionadas que superan un umbral. La probabilidad de cada patrón de diagnósticos es la probabilidad de un ortante de una normal multivariada, que no tiene solución analítica y hay que aproximar.
2. Inferencia basada en simulaciones (SBI): entrenar una red neuronal que aproxime el posterior a partir de muchas simulaciones del mismo simulador.

En ambos casos, comparar las estimaciones con las del modelo Dendukuri para todos los pares en los dos escenarios.

4. Entrega

En su repositorio suban:

- Sección 1: el código del pasaje de mensajes y sus figuras.

- Sección 2: la especificación del modelo alternativo y la justificación de sus supuestos. Si lo implementan, su versión de `comparacion_modelos.py`, la salida que imprime y las figuras.
- Sección 3: el código de cada modelo y las figuras de valores verdaderos contra posteriors.
- Un archivo `practica_2_2.md` con las figuras en PNG incrustadas y las respuestas a cada objetivo.

No suban los datos simulados (`.npz`), que se vuelven a generar con `simulate.py` y `simulate_scenarios.py`.

Referencias

[1] Cheung A, Dufour S, Jones G, Kostoulas P, Stevenson M, Singanallur N, et al. Bayesian latent class analysis when the reference test is imperfect. Revue Scientifique et Technique (International Office of Epizootics). 2021;40(1):271–286.

[2] Umemneku Chikere CM, Wilson K, Graziadio S, Vale L, Allen AJ. Diagnostic test evaluation methodology: a systematic review of methods employed to evaluate diagnostic tests in the absence of gold standard–an update. PLoS One. 2019;14(10):e0223832.

[3] Dendukuri N, Joseph L. Bayesian approaches to modeling the conditional dependence between multiple diagnostic tests. Biometrics. 2001;57(1):158–167.

[4] Georgiadis MP, Johnson WO, Gardner IA, Singh R. Correlation-adjusted estimation of sensitivity and specificity of two diagnostic tests. Journal of the Royal Statistical Society Series C: Applied Statistics. 2003;52(1):63–76.

[5] Hui SL, Walter SD. Estimating the error rates of diagnostic tests. Biometrics. 1980; p. 167–171.

[6] Vacek PM. The effect of conditional dependence on the evaluation of diagnostic tests. Biometrics. 1985; p. 959–968.

[7] Branscum A, Gardner I, Johnson W. Estimation of diagnostic-test sensitivity and specificity through Bayesian modeling. Preventive veterinary medicine. 2005;68(2-4):145–163.